

End-to-End MLOps Pipeline for Heart Disease Risk Prediction

AIMLCZG523 - Machine Learning Operations
Assignment 01

Student	SATHISHKUMAR P
BITS ID	2024AD05172
Repository	https://github.com/SathishKumarPatchaiappan/heart-disease-mlops-assignment
Submission Type	End-to-end MLOps implementation report

This report describes the actual implementation I built and tested on my local Windows machine. I used a practical MLOps stack - MLflow, FastAPI, GitHub Actions, Docker, and Docker Desktop Kubernetes - so that the work could be reproduced without needing a paid cloud environment.

Dataset UCI Cleveland	Training Scikit-learn + MLflow	API FastAPI	Container Docker	Deployment Kubernetes + Metrics
->	->	->	->	monitored service

Figure 1. High-level workflow used for the assignment implementation.

Contents

1. Executive summary and scope
2. Dataset, cleaning, and exploratory analysis
3. Feature engineering and model development
4. Evaluation and MLflow tracking
5. Reproducibility, testing, and CI/CD
6. API serving and Docker containerization
7. Kubernetes deployment
8. Monitoring and logging
9. Limitations, ethics, and future work
10. Reproduction commands and evidence

1. Executive Summary and Scope

For this assignment, I built a complete MLOps pipeline for heart disease risk prediction using the UCI Cleveland heart disease dataset. I treated the work as a small production-style project rather than only a notebook exercise. The implementation covers data preparation, model training, experiment tracking, API serving, automated testing, Docker packaging, Kubernetes deployment, and basic monitoring.

Two models were trained and compared: Logistic Regression and Random Forest. Logistic Regression was selected because it produced the better ROC-AUC score while remaining simple and easy to explain. The final model was packaged as a reusable Scikit-learn pipeline so the same preprocessing steps are used during both training and inference.

The final model comparison is shown below:

Model	ROC-AUC	F1-score
Logistic Regression	0.966	0.881
Random Forest	0.948	0.881

Table 1. Model comparison after tuning and evaluation.

The API was successfully tested in three ways: directly through FastAPI, inside a Docker container, and through a Kubernetes service using Docker Desktop Kubernetes. This gives evidence that the model can be served in an isolated environment and moved into an orchestration layer.

2. Dataset, Cleaning, and Exploratory Analysis

The dataset used in this work is the UCI Heart Disease Cleveland processed dataset. It has 303 rows, 13 input features, and one target variable. The original target values range from 0 to 4. I converted the target into a binary classification problem, where 0 represents no heart disease and 1 represents the presence of heart disease. This transformation makes the problem suitable for a risk prediction API.

The raw data is stored under `data/raw/processed.cleveland.data`, and the cleaned version is stored as `data/processed/heart_disease_clean.csv`. During cleaning, the main missing values were found in the `ca` and `thal` columns. I did not manually hard-code replacements in the raw data. Instead, I handled missing values inside the preprocessing pipeline so that training and inference follow the same logic.

The exploratory data analysis covered missing values, class distribution, numerical histograms, a correlation heatmap, and target-based feature comparisons. The class split was reasonably balanced, and features such as chest pain type, `thalach`, `oldpeak`, exercise-induced angina, and `thalassemia` showed useful relationship with the target.

3. Feature Engineering and Model Development

I used a Scikit-learn `ColumnTransformer` as the central preprocessing component. Numerical variables were imputed with median values and scaled using `StandardScaler`. Categorical variables were imputed with the most frequent value and encoded using `OneHotEncoder`. This approach keeps the preprocessing reusable and avoids train/test leakage.

The model training script performs a stratified train/test split so that the target distribution is preserved. I trained Logistic Regression and Random Forest because they provide a useful comparison between a linear baseline and a non-linear ensemble model. `GridSearchCV` was used for basic hyperparameter tuning, and the final metrics were computed on the test data.

The trained model was saved as `models/heart_disease_pipeline.joblib`. Since the artifact is a full pipeline, the API does not need to repeat preprocessing code manually. It only loads the pipeline and sends incoming patient features into it.

4. Evaluation and MLflow Tracking

I used accuracy, precision, recall, F1-score, and ROC-AUC for model evaluation. ROC-AUC was used as the main selection metric because this is a binary medical-risk classification problem, where ranking ability and threshold flexibility are important.

MLflow was used to track the training runs. The experiment name was heart-disease-classification. For each run, the code logged model parameters, metrics, ROC curves, confusion matrices, and the trained model artifact. This made it easy to compare the Logistic Regression and Random Forest runs side by side.

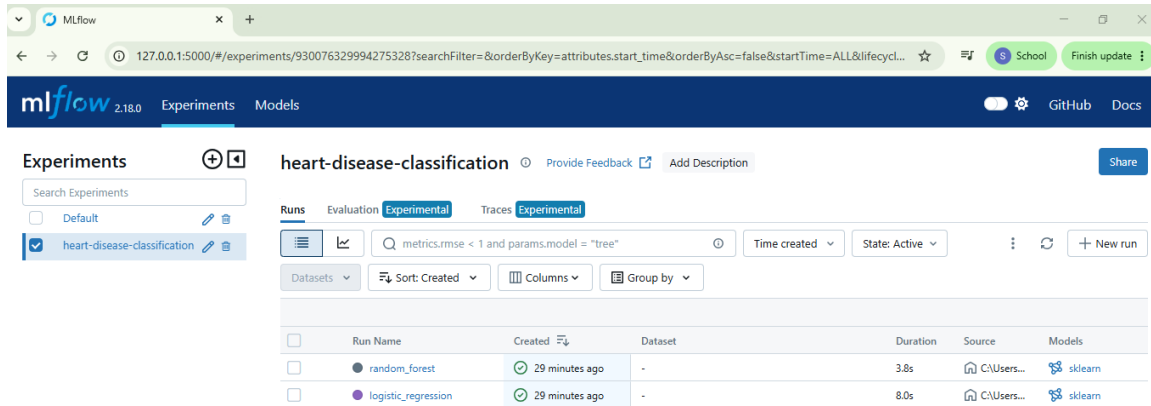


Figure 2. MLflow experiment containing both model runs.

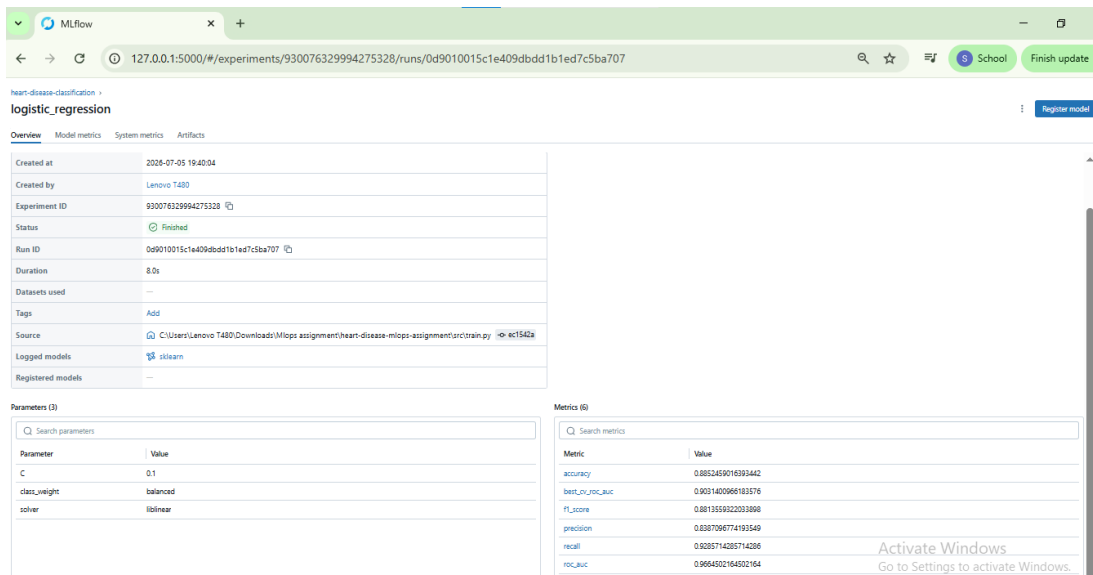


Figure 3. Logistic Regression run details with parameters and metrics.

5. Reproducibility, Testing, and CI/CD

The project is structured as a reproducible repository rather than a single notebook. It includes source code, tests, configuration, Docker and Kubernetes files, generated figures, screenshots, and the final report. Dependencies are listed in requirements.txt, and the training can be rerun using `python -m src.train`.

I added unit tests for the data loading, model artifact, and API endpoints. The local test run passed with five tests. I also used Ruff for code quality checks, and all linting checks passed. GitHub Actions was used as the CI/CD pipeline. The workflow validates the repository automatically by installing dependencies, running linting, running tests, and checking build-related steps.

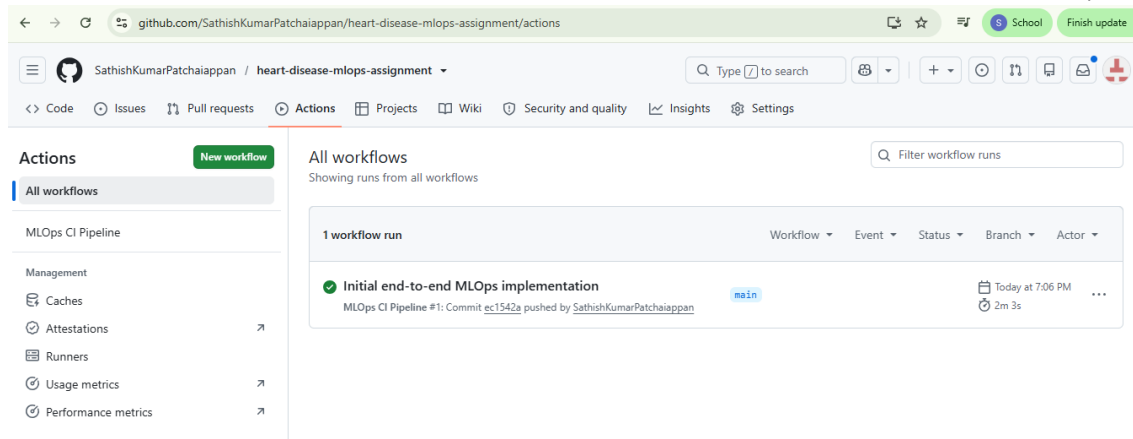


Figure 4. GitHub Actions workflow completed successfully.

```

(.venv) PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-8.3.3, pluggy-1.6.0
rootdir: C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.1
collected 5 items

tests\test_api.py .. [ 40%]
tests\test_data.py .. [ 80%]
tests\test_model.py . [100%]

===== warnings summary =====
tests\test_api.py::test_health_and_prediction
  C:\Users\Lenovo T480\Downloads\Mlops assignment\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:455: OptimizeWarning:
Unknown solver options: iprint
  opt_res = optimize.minimize(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 3 passed, 1 warning in 1.03s =====
(.venv) PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment>
  
```

Figure 5. Local Pytest result showing five tests passed.

```

(.venv) PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.12.4, pytest-8.3.3, pluggy-1.6.0
rootdir: C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.14.1
collected 5 items

tests\test_api.py .. [ 40%]
tests\test_data.py .. [ 80%]
tests\test_model.py . [100%]

===== warnings summary =====
tests\test_api.py::test_health_and_prediction
  C:\Users\Lenovo T480\Downloads\Mlops assignment\.venv\Lib\site-packages\sklearn\linear_model\_logistic.py:455: OptimizeWarning:
Unknown solver options: iprint
  opt_res = optimize.minimize(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 3 passed, 1 warning in 1.03s =====
(.venv) PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment> python -m ruff check .
All checks passed!
(.venv) PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment>
  
```

Figure 6. Ruff code-quality check completed with all checks passed.

6. API Serving and Docker Containerization

FastAPI was used to serve the model because it provides a clean Swagger UI, request validation, and simple JSON-based endpoints. The API exposes four endpoints: /, /health, /predict, and /metrics. The /predict endpoint accepts patient feature values and returns a binary prediction, a risk label, confidence, and model version.

```

{
  "age": 63, "sex": 1, "cp": 1, "trestbps": 145,
  "chol": 233, "fbs": 1, "restecg": 2, "thalach": 150,
  "exang": 0, "oldpeak": 2.3, "slope": 3, "ca": 0, "thal": 6
}
  
```

To make the service portable, I created a Docker image containing the API code, dependencies, and trained model artifact. The container was run locally on port 8001 and tested successfully through Swagger.

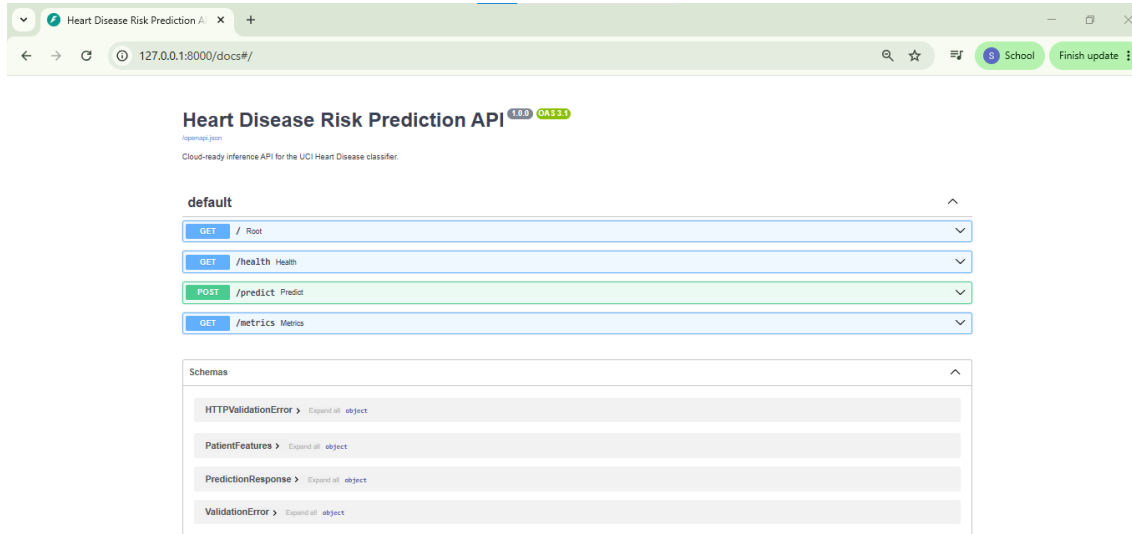


Figure 7. FastAPI Swagger UI showing the available endpoints.

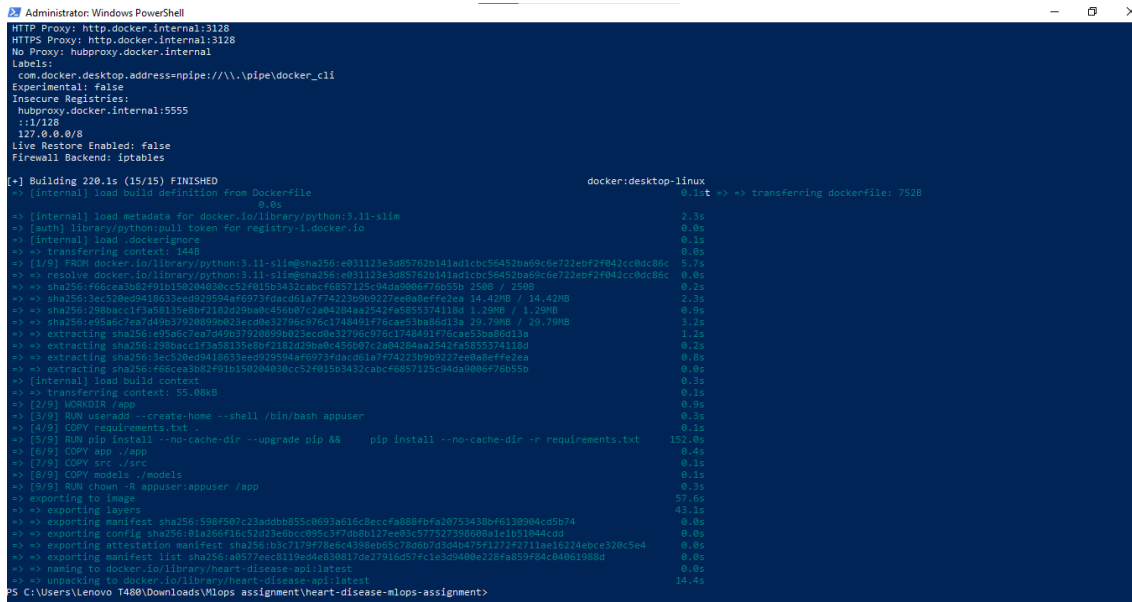


Figure 8. Docker image build completed successfully.

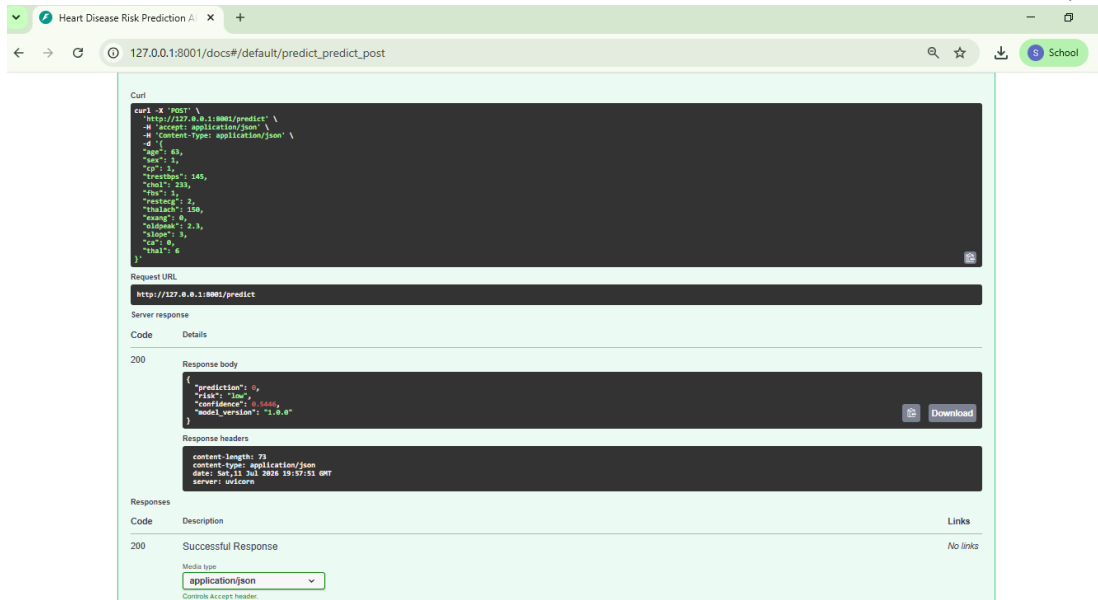


Figure 9. Docker-hosted /predict endpoint returning a successful response.

7. Kubernetes Deployment

For deployment, I used Docker Desktop Kubernetes. This was a practical choice because the assignment allows local Kubernetes, and it avoids the cost and setup time of a public cloud cluster. The deployment folder contains the Kubernetes ConfigMap, Deployment, and Service manifests.

The Docker image was tagged as `heart-disease-api:1.0.0` and deployed using `kubectl apply -f deployment/`. The deployment created two running pods and a service. Direct NodePort access was not reachable from localhost on my machine, so I used `kubectl port-forward service/heart-disease-api-service 8080:80` to test the service reliably.

```
PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
heart-disease-api-5f98fbc88d-bkc46  1/1     Running   0           106s
heart-disease-api-5f98fbc88d-ck91l  1/1     Running   0           106s
PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment> kubectl get deployments
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
heart-disease-api                   2/2     2             2           106s
PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment> kubectl get svc
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
heart-disease-api-service           NodePort    10.96.163.123 <none>        80:30080/TCP     109s
kubernetes                         ClusterIP   10.96.0.1    <none>        443/TCP          5m7s
PS C:\Users\Lenovo T480\Downloads\Mlops assignment\heart-disease-mlops-assignment>
```

Figure 10. Kubernetes pods, deployment, and service are running.

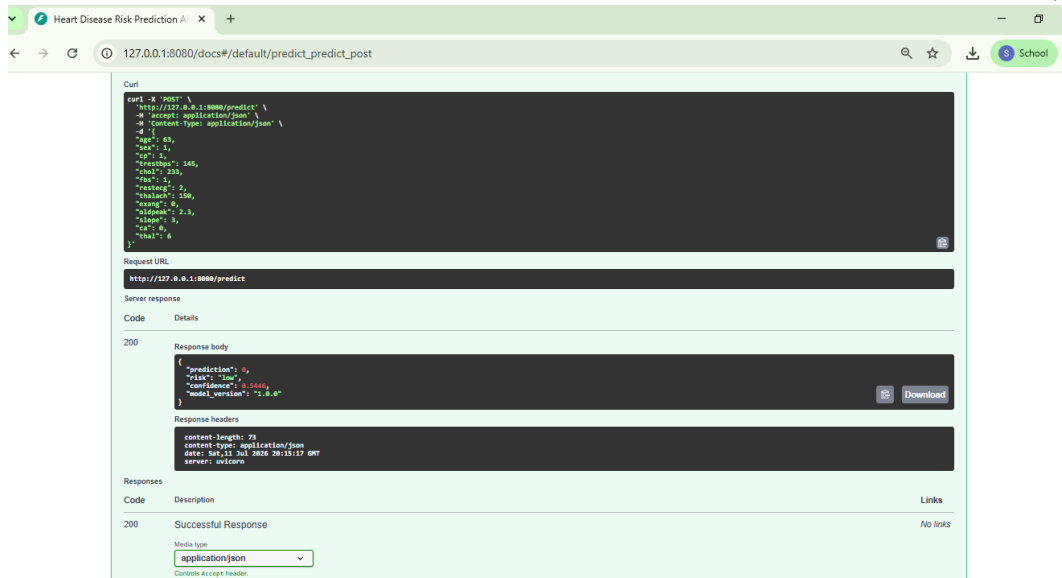


Figure 11. Kubernetes-hosted /predict endpoint tested through port forwarding.

8. Monitoring and Logging

The API exposes Prometheus-compatible metrics through the /metrics endpoint. I tracked request counts, endpoint-level status codes, request latency, and prediction counts. These metrics are basic, but they are enough to show that the service can be monitored after deployment.

The service also writes runtime logs for API requests and prediction outputs. In a real production environment, I would forward these logs to a centralized platform and add alerts for high latency, repeated errors, or unusual prediction patterns.

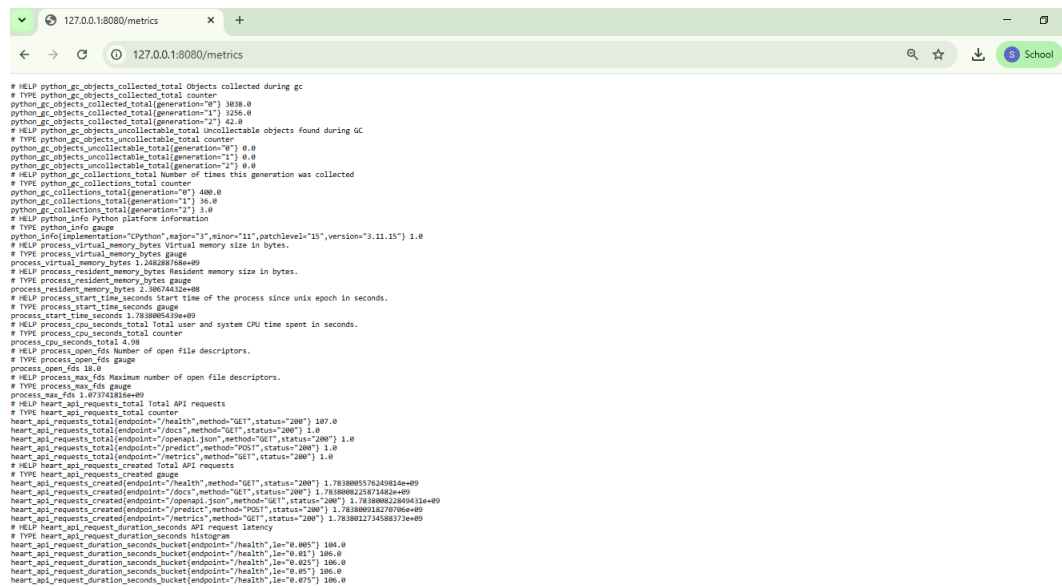


Figure 12. Kubernetes metrics endpoint showing API request and prediction counters.

9. Limitations, Ethics, and Future Work

This project is for academic learning and should not be treated as a real medical diagnosis system. The dataset is small, historical, and may not represent current patient populations. It also does not include many clinical factors that doctors would normally consider.

From an ethics perspective, the model should only support decision-making and should not replace clinical judgement. If this were developed further, I would add model calibration, data drift monitoring, authentication, input validation rules agreed with domain experts, and a retraining workflow. I would also use a larger and more recent dataset before considering any real-world usage.

10. Reproduction Commands

The repository can be reproduced using the following commands. I have kept the commands close to what I actually used during implementation and testing.

```
git clone https://github.com/SathishKumarPatchaiappan/heart-disease-mlops-assignment.git
cd heart-disease-mlops-assignment
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pytest -v
python -m ruff check .
python -m src.train
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
docker build -t heart-disease-api:latest .
docker run -p 8001:8000 heart-disease-api:latest
docker tag heart-disease-api:latest heart-disease-api:1.0.0
kubectl apply -f deployment/
kubectl get pods
kubectl get deployments
kubectl get svc
kubectl port-forward service/heart-disease-api-service 8080:80
```

11. Evidence Checklist

The screenshots below were captured during the actual implementation and are stored in reports/screenshots in the GitHub repository.

Screenshot	Evidence shown
01_github_actions_passed.png	GitHub Actions CI pipeline passed
02_local_pytest_passed.png	Local unit tests passed
03_ruff_checks_passed.png	Ruff code quality check passed
05_mlflow_experiment_runs.png	MLflow experiment runs created
06_mlflow_run_metrics.png	Model metrics logged in MLflow
13_fastapi_swagger_ui.png	FastAPI Swagger UI
15_docker_build_success.png	Docker image build success
17_docker_prediction_success.png	Docker prediction success
20_kubernetes_pods_services_ready.png	Kubernetes pods and services ready
22_kubernetes_prediction_success.png	Kubernetes prediction success
24_kubernetes_metrics_endpoint.png	Kubernetes metrics endpoint

12. Conclusion

This assignment helped me connect the full MLOps chain rather than focusing only on model accuracy. The final solution includes a trained model, reusable preprocessing pipeline, experiment tracking, tests, CI/CD, API serving, Docker packaging, Kubernetes deployment, and monitoring evidence. The repository contains the code, artifacts, deployment files, screenshots, and report needed to review and reproduce the work.

Repository: <https://github.com/SathishKumarPatchaiappan/heart-disease-mlops-assignment>